# SHADOWMEM — CONFIG & LOGGING DEEP AUDIT REPORT

Direct Codebase Verification: core/config.py, core/logging.py, Cross-Check Usage & Live Overrides

**Audit Date:** October 11, 2026 - 16:33 | **Target Files:** core/config.py, core/logging.py, detecter.py | **Auditor:** Security Engineering

## 1. Audit Scorecard & Executive Findings

| Check # | Verification Requirement | Observed Implementation State | Verdict |
|---|---|---|---|
| **Check 1** | core/config.py definitions & pydantic-settings BaseSettings | Contains all 19 configuration fields and .env / SHADOWMEM_* parsing, but inherits from **pydantic.BaseModel** rather than pydantic_settings.BaseSettings because pydantic-settings is not installed. | **FAIL** |
| **Check 2** | core/logging.py real structured logger setup | Configures formal root and app loggers, dedicated StreamHandler(sys.stdout), custom Formatter, and duplicate handler suppression. | **PASS** |
| **Check 3** | Codebase cross-check for hardcoded values | Production files (app.py, services/, routers/) are migrated; however, **detecter.py**, memoryscanner.py, and test scripts still contain hardcoded values. | **FAIL** |
| **Check 4** | Zero leftover duplicate definitions | **detecter.py** retains duplicate inline copies of JIT_ENGINE_ALLOWLIST, MANAGED_RUNTIME_NAMES, JIT_SCORE_CAP (25), and thresholds (70/35). | **FAIL** |
| **Check 5** | Live app startup with .env & env var override | Server dynamically bound to port **8765** via SHADOWMEM_PORT and port **8877** via .env file, returning HTTP 200 OK responses. | **PASS** |

## 2. Check 1 Analysis: core/config.py Implementation

**Direct Inspection Finding:** core/config.py implements a comprehensive configuration catalog covering 19 parameters across server, forensics, alert streaming, and future extensions. However, it **FAILS** the strict requirement to inherit from pydantic_settings.BaseSettings.

• **BaseModel vs. BaseSettings:** The class is defined as class Settings(BaseModel):. The reason is that the pydantic-settings package is not installed in the virtual environment (only pydantic 2.13.5 is present). To avoid a runtime crash, the module was implemented using standard Pydantic 2.x BaseModel coupled with custom loader functions.
• **.env Loading Mechanism:** Implemented in _load_env_file() (lines 15–38), which parses the repo root .env file on initial module import and injects values into os.environ.
• **Environment Variable Overrides:** Implemented in _get_env() (lines 45–52) and Settings.load() (lines 191–236), which searches for SHADOWMEM_<KEY> overrides.

## Catalog of All 19 Configuration Parameters in core/config.py:

| Parameter Name | Type | Default Value | SHADOWMEM_ Override |
|---|---|---|---|
| app_title / app_version | `str` | `'ShadowMem' / '1.0.0'` | `SHADOWMEM_APP_TITLE` |
| host / port | `str / int` | `'0.0.0.0' / 8000` | `SHADOWMEM_HOST / PORT` |
| reload / api_prefix | `bool / str` | `False / '/api/v1'` | `SHADOWMEM_RELOAD / API_PREFIX` |
| cors_origins | `List[str]` | `['*']` | `SHADOWMEM_CORS_ORIGINS` |
| frontend_dir | `Path` | `frontend/out` | `Default path relative to root` |
| model_path | `Path` | `shadowmem_model.pkl` | `SHADOWMEM_MODEL_PATH` |
| threshold_malicious | `float` | `70.0` | `SHADOWMEM_THRESHOLD_MALICIOUS` |
| threshold_suspicious | `float` | `35.0` | `SHADOWMEM_THRESHOLD_SUSPICIOUS` |
| jit_score_cap | `float` | `25.0` | `SHADOWMEM_JIT_SCORE_CAP` |
| jit_engine_allowlist | `Set[str]` | 12 runtimes (chrome, msedge, code...) | `SHADOWMEM_JIT_ENGINE_ALLOWLIST` |
| managed_runtime_names | `Set[str]` | `{powershell, pwsh, dotnet}` | `Hardcoded default set` |
| managed_runtime_path_prefixes | `Tuple[str]` | `('c:\windows\',)` | `Hardcoded default tuple` |
| critical_system_processes | `Set[str]` | 13 OS binaries (csrss, lsass, smss...) | `Hardcoded default set` |
| scan_interval_seconds | `float` | `30.0` | `SHADOWMEM_SCAN_INTERVAL_SECONDS` |
| threat_statuses | `Tuple[str]` | ('Malicious (Injected)', 'Suspicious') | `Hardcoded default tuple` |
| max_region_details | `int` | `16` | `Hardcoded default integer` |
| log_level / log_format | `str / str` | `'INFO' / Timestamped format` | `SHADOWMEM_LOG_LEVEL` |
| docker_sandbox_enabled | `bool` | `False` | `SHADOWMEM_DOCKER_SANDBOX_ENABLED` |
| docker_sandbox_wsl_distro | `str` | `'Ubuntu'` | `SHADOWMEM_DOCKER_WSL_DISTRO` |
| supply_chain_scanner_enabled | `bool` | `False` | `SHADOWMEM_SUPPLY_CHAIN_SCANNER_ENABLED` |

## 3. Check 2 Analysis: core/logging.py Structured Logger

**Direct Inspection Finding: PASS.** core/logging.py sets up a real structured logging pipeline rather than a bare logging.basicConfig() one-liner.
• **Custom Stream Handler:** Explicitly instantiates a logging.StreamHandler(sys.stdout) and binds a uniform timestamped formatter: %(asctime)s [ShadowMem] %(levelname)s [%(name)s] %(message)s.
• **Duplicate Handler Prevention:** Inspects root_logger.handlers. If handlers already exist (e.g. during Uvicorn reload or secondary module imports), it updates their level and format rather than stacking duplicate handlers that cause doubled log lines.
• **Logger Namespace:** Exposes logger = logging.getLogger('shadowmem') and get_logger(name).

### Complete Code of core/logging.py (50 lines):

```
import logging
import sys
from typing import Optional
from core.config import settings

def setup_logging(log_level: Optional[str] = None, log_format: Optional[str] = None) -> logging.Logger:
    """Configure and return the root ShadowMem logger."""
    level_name = (log_level or settings.log_level).upper()
    level = getattr(logging, level_name, logging.INFO)
    fmt = log_format or settings.log_format

    root_logger = logging.getLogger()
    root_logger.setLevel(level)

    if not root_logger.handlers:
        handler = logging.StreamHandler(sys.stdout)
        handler.setLevel(level)
        formatter = logging.Formatter(fmt)
        handler.setFormatter(formatter)
        root_logger.addHandler(handler)
    else:
        for handler in root_logger.handlers:
            handler.setLevel(level)
            handler.setFormatter(logging.Formatter(fmt))

    app_logger = logging.getLogger("shadowmem")
    app_logger.setLevel(level)
    return app_logger

def get_logger(name: str = "shadowmem") -> logging.Logger:
    return logging.getLogger(name)

logger = setup_logging()
```

## 4. Checks 3 & 4 Analysis: Codebase Usage & Duplicate Definitions

**Direct Inspection Finding: FAIL.** While all primary server and service files were successfully migrated to import from core/config.py, standalone tools and test harnesses were **NOT** migrated, resulting in leftover hardcoded values and duplicate definitions across the repository.

## Inventory of Unmigrated Files with Hardcoded Inline Values:

| File Path | Line Numbers | Hardcoded Values / Definitions | Architectural Impact & Risk |
|---|---|---|---|
| **detecter.py** (CLI Detection Tool) | Lines 8, 11–24, 26, 31–32, 111, 114 | • MODEL_PATH = 'shadowmem_model.pkl'<br>• JIT_ENGINE_ALLOWLIST = { ... } (13 browsers/tools)<br>• JIT_SCORE_CAP = 25<br>• MANAGED_RUNTIME_NAMES / PATH_PREFIXES<br>• threat_score >= 70 and >= 35 (thresholds) | **High Divergence Risk:** CLI detection tool operates on hardcoded constants and does not respect .env or SHADOWMEM_* overrides. |
| **memoryscanner.py** | Line 33 | `_MAX_REGION_DETAILS = 64` | Inconsistent cap: config.py defines max_region_details = 16, while memoryscanner.py caps at 64. |
| **train_model.py** | Line 605 | `output_path = 'shadowmem_model.pkl'` | Hardcoded artifact output path instead of referencing settings.model_path. |
| **malware_simulator.py** | Lines 119–121 | `PROTOTYPE_URL = 'http://localhost:8000'`<br>`WEBSOCKET_URL = 'ws://localhost:8000/ws/alerts'` | Test harness cannot target dynamic ports configured via SHADOWMEM_PORT without manual file edits. |
| **tests/mocks/server.py** | Lines 31, 290 | `port = 8082` | Expected mock port, but hardcoded inline instead of via test configuration. |
| **launch_shadowmem.bat** | Lines 14, 17 | `http://127.0.0.1:8000 and --port 8000` | Windows launcher batch script does not inspect .env file before launching browser. |

## 5. Check 5 Analysis: Live Startup & Port Override Proof

**Direct Inspection Finding: PASS.** The server was executed in live background processes under two distinct override conditions to prove that configuration loading is fully operational and binds dynamically.

### Test A: Environment Variable Override (SHADOWMEM_PORT=8765)

```
[COMMAND] python app.py  (with env SHADOWMEM_PORT=8765)
[UVICORN LOGS]
  INFO:     Started server process [3108]
  INFO:     Waiting for application startup.
  INFO:     Application startup complete.
  INFO:     Uvicorn running on http://0.0.0.0:8765 (Press CTRL+C to quit)

[HTTP REQUEST] GET http://127.0.0.1:8765/api/v1/sandbox/health
[HTTP STATUS]  200 OK
[RESPONSE BODY]
  {
    "enabled": false,
    "wsl_distro": "Ubuntu",
    "wsl_available": true,
    "docker_ready": false,
    "message": "Docker sandbox is disabled in configuration (SHADOWMEM_DOCKER_SANDBOX_ENABLED=false)."
  }
```

### Test B: .env File Loading (SHADOWMEM_PORT=8877, SHADOWMEM_HOST=127.0.0.1)

```
[FILE CREATED] .env in repository root:
  SHADOWMEM_PORT=8877
  SHADOWMEM_HOST=127.0.0.1

[COMMAND] python app.py
[UVICORN LOGS]
  INFO:     Started server process [11976]
  INFO:     Waiting for application startup.
  INFO:     Application startup complete.
  INFO:     Uvicorn running on http://127.0.0.1:8877 (Press CTRL+C to quit)

[HTTP REQUEST] GET http://127.0.0.1:8877/api/v1/supply-chain/health
[HTTP STATUS]  200 OK
[RESPONSE BODY]
  {
    "enabled": true,
    "pefile_available": true,
    "message": "Supply chain and module integrity scanner is active."
  }
```

## 6. Remediation Plan to Achieve 100% Pass

To resolve the three failed checks and bring configuration management to complete consistency:

1. **Install pydantic-settings and Refactor Settings (Fixes Check 1):**
Execute pip install pydantic-settings and update core/config.py to use from pydantic_settings import BaseSettings, SettingsConfigDict with model_config = SettingsConfigDict(env_prefix='SHADOWMEM_', env_file='.env', extra='ignore').
2. **Migrate detecter.py to Import from core/config.py (Fixes Checks 3 & 4):**
Replace hardcoded JIT_ENGINE_ALLOWLIST, MANAGED_RUNTIME_NAMES, JIT_SCORE_CAP, and thresholds 70 / 35 in detecter.py with from core.config import settings and reference settings.threshold_malicious, etc.
3. **Synchronize _MAX_REGION_DETAILS in memoryscanner.py (Fixes Check 3):**
Bind _MAX_REGION_DETAILS to settings.max_region_details.
4. **Update malware_simulator.py & Launcher Scripts (Fixes Check 3):**
Allow malware_simulator.py to read target port from settings.port rather than hardcoding port 8000.